Section 1.2: Logistic Regression

AI Class — Lecture Notes

1 From Regression to Classification

In Section 1.1 we predicted a *continuous* output $y \in \mathbb{R}$. Now suppose the output is a **binary label**: $y \in \{0, 1\}$. For example, we might ask:

Input x	**Label** y
Tumor size (cm)	0 = benign, 1 = malignant
Hours studied	0 = fail, 1 = pass
Email features	0 = not spam, 1 = spam

1.1 A Toy Example

Consider six observations, each labelled either class F or class T. We encode the labels numerically as $y = 0$ for F and $y = 1$ for T:

x	1	2	3	4	5	6
Class	F	F	T	F	T	T
y	0	0	1	0	1	1

Small x is mostly F and large x is mostly T, but the classes *overlap* around $x = 3$ and $x = 4$. This is the region where we should be *uncertain* about the label.

A naive attempt: linear regression. Since y is now a number, nothing stops us from fitting $\hat{y} = wx + b$ with the closed-form solution from Section 1.1. Here $\bar{x} = 3.5$ and $\bar{y} = 0.5$:

i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
1	-2.5	-0.5	1.25	6.25
2	-1.5	-0.5	0.75	2.25
3	-0.5	0.5	-0.25	0.25
4	0.5	-0.5	-0.25	0.25
5	1.5	0.5	0.75	2.25
6	2.5	0.5	1.25	6.25
		Sum	3.5	17.5

$$w = \frac{3.5}{17.5} = 0.2, \qquad b = 0.5 - 0.2 \times 3.5 = -0.2.$$

The fitted line is $\hat{y} = 0.2\,x - 0.2$.

Why not use linear regression? As a *classifier*, the line is not terrible: thresholding at $\hat{y} = 0.5$ gives the boundary $x = 3.5$, right in the middle of the overlap. The trouble starts when we try to read $\hat{y}$ as a *probability* of class T (see the accompanying marimo notebook):

- **Values outside $[0, 1]$.** At $x = 0$ the line predicts $\hat{y} = -0.2$, and at $x = 8$ it predicts $\hat{y} = 1.4$. Neither is a valid probability.
- **Constant slope.** Each extra unit of x adds the same 0.2, whether we are deep inside one class or in the uncertain overlap. Intuitively, moving from $x = 3$ to $x = 4$ should change our belief a lot, while moving from $x = 5$ to $x = 6$ (already confidently T) should change it very little.
- **Sensitivity to outliers.** A single far-away point, e.g. $(x, y) = (20, 1)$, flattens the line and drags the 0.5-threshold to the right, so points that were correctly classified before can become misclassified.

1.2 What Shape Do We Want?

What we really want is a model that outputs a *probability*:

$$\hat{p} = P(y = 1 \mid x) \in (0, 1).$$

For the toy data, a sensible curve should:

- stay close to 0 for small x (confidently F),
- stay close to 1 for large x (confidently T),
- rise *smoothly* through the overlap region, passing 0.5 near $x = 3.5$, where we are least certain.

This is an **S-shaped** curve (see the marimo notebook), not a straight line.

2 The Sigmoid Function: Motivation via Log-Odds

We would still like to use a linear function $wx + b$: it is simple, interpretable, and we already know how to work with it. The problem is a mismatch of ranges:

$$wx + b \in (-\infty, \infty) \qquad \text{but} \qquad p \in (0, 1).$$

So instead of forcing p itself to be linear in x, we **transform** p into a quantity that lives on all of $\mathbb{R}$, and make *that* quantity linear. We do this in two steps.

Step 1: probability $\to$ odds. The **odds** of an event with probability p are:

$$\text{odds} = \frac{p}{1 - p} \in (0, \infty).$$

This removes the upper limit of 1, but the odds are still positive.

Step 2: odds $\to$ log-odds. Taking the logarithm maps $(0, \infty)$ onto all of $\mathbb{R}$:

$$\text{log-odds} = \log \frac{p}{1 - p} \in (-\infty, \infty).$$

This quantity is also called the **logit** of p.

p	odds $= \dfrac{p}{1-p}$	log-odds	Interpretation
0.1	1/9	-2.20	likely F
0.5	1	0	completely unsure
0.9	9	2.20	likely T

The log-odds are symmetric around 0, and $p = 0.5$ maps exactly to 0. Now the ranges match, so we model the log-odds as linear in x:

$$\log \frac{p}{1-p} = wx + b. \tag{1}$$

The log-odds is not the only possible choice: any function that maps $(0, 1)$ onto $\mathbb{R}$ would work (for example, the inverse of the normal CDF gives *probit* regression). The log-odds is preferred because it is simple and interpretable: increasing x by one unit multiplies the odds by e^w.

2.1 Solving for p

Exponentiating both sides of (1) and solving for p:

$$\begin{aligned} \frac{p}{1-p} &= e^{wx+b} \\ p &= (1-p)\, e^{wx+b} \\ p\,(1 + e^{wx+b}) &= e^{wx+b} \end{aligned}$$

$$\hat{p} \;=\; \frac{1}{1+e^{-(wx+b)}} \;=\; \sigma(wx+b)$$

This function $\sigma(z) = \dfrac{1}{1+e^{-z}}$ is called the **sigmoid** (or logistic) function. It is exactly the smooth S-shaped curve we asked for: by making the log-odds linear, the probability automatically becomes S-shaped in x.

Key properties of $\sigma(z)$.

- Output is always in $(0, 1)$ — valid as a probability.
- $\sigma(0) = 0.5$: the decision boundary is where $wx + b = 0$.
- As $z \to +\infty$, $\sigma(z) \to 1$; as $z \to -\infty$, $\sigma(z) \to 0$.
- Its derivative has the elegant form $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$ (see Appendix A).

3 The Logistic Regression Model

For d features, the model generalises directly:

$$\hat{p}^{(i)} = \sigma\big(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}\big) = \frac{1}{1 + e^{-\boldsymbol{\theta}^\top \mathbf{x}^{(i)}}}, \tag{2}$$

where $\boldsymbol{\theta} = [b,\, w_1,\, \ldots,\, w_d]^\top$ and $\mathbf{x}^{(i)}$ includes a leading 1 for the bias (same design-matrix convention as Section 1.1).

The predicted class is:

$$\hat{y}^{(i)} = \begin{cases} 1 & \text{if } \hat{p}^{(i)} \geq 0.5 \\ 0 & \text{otherwise.} \end{cases}$$

4 Cost Function: Cross-Entropy via MLE

4.1 Setting Up the Likelihood

We model each label $y^{(i)}$ as a Bernoulli random variable with success probability $\hat{p}^{(i)}$:

$$P\big(y^{(i)} \mid \mathbf{x}^{(i)};\, \boldsymbol{\theta}\big) = \big(\hat{p}^{(i)}\big)^{y^{(i)}} \big(1 - \hat{p}^{(i)}\big)^{1 - y^{(i)}}.$$

Assuming the n observations are independent, the joint likelihood is:

$$\mathcal{L}(\boldsymbol{\theta}) = \prod_{i=1}^{n} \big(\hat{p}^{(i)}\big)^{y^{(i)}} \big(1 - \hat{p}^{(i)}\big)^{1 - y^{(i)}}.$$

4.2 From MLE to the Cross-Entropy Loss

Maximising $\mathcal{L}$ is equivalent to maximising the log-likelihood $\ell = \log \mathcal{L}$ (log is monotone):

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{n} \Big[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log\big(1 - \hat{p}^{(i)}\big) \Big].$$

Minimising the **negative** log-likelihood (and averaging over n) gives the **Binary Cross-Entropy (BCE)** loss:

$$J(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^{n} \Big[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log\big(1 - \hat{p}^{(i)}\big) \Big]$$

Why cross-entropy? When $y^{(i)} = 1$, only the first term $-\log \hat{p}^{(i)}$ matters: we are penalised heavily if $\hat{p}^{(i)}$ is close to 0 (confident and wrong). When $y^{(i)} = 0$, only the second term $-\log(1 - \hat{p}^{(i)})$ matters. The two terms switch on and off depending on the true label.

5 The Gradient and Why We Need Gradient Descent

Unlike MSE in linear regression, there is **no closed-form solution** for the minimiser of $J(\boldsymbol{\theta})$. The cross-entropy loss is convex (it has a unique global minimum), but we cannot invert the normal equations here because the sigmoid is non-linear.

Taking the gradient with respect to $\boldsymbol{\theta}$ (using the chain rule and the identity $\sigma'(z) = \sigma(z)(1-\sigma(z))$):

$$\nabla_{\boldsymbol{\theta}} J = \frac{1}{n} \sum_{i=1}^{n} \left(\hat{p}^{(i)} - y^{(i)} \right) \mathbf{x}^{(i)}$$

Remarkably clean form. The gradient has exactly the same structure as in linear regression — it is a weighted sum of the input vectors, where the weights are the *prediction errors* $(\hat{p}^{(i)} - y^{(i)})$. The sigmoid "absorbed" all the complexity through the chain rule.

Because there is no closed form, we use **gradient descent**: starting from an initial $\boldsymbol{\theta}^{(0)}$, we repeatedly update

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J,$$

where $\alpha > 0$ is the **learning rate**. Gradient descent will be covered in detail in the next section.

6 Summary

Model	$\hat{p} = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$
Loss function	BCE: $J = -\frac{1}{n} \sum \left[y \log \hat{p} + (1-y) \log(1-\hat{p}) \right]$
Gradient	$\nabla_{\boldsymbol{\theta}} J = \frac{1}{n} \sum (\hat{p}^{(i)} - y^{(i)}) \, \mathbf{x}^{(i)}$
Solution method	Gradient descent (no closed form)
Works well when	The decision boundary is (approximately) linear

Up next (Section 1.3): What if no linear boundary can separate the classes? We revisit the XOR dataset to show that both linear regression and logistic regression fail — motivating the need for non-linearity and neural networks.

A Derivative of the Sigmoid Function

Let $\sigma(z) = (1 + e^{-z})^{-1}$. By the chain rule:

$$\sigma'(z) = -\frac{-e^{-z}}{(1+e^{-z})^2} = \frac{e^{-z}}{(1+e^{-z})^2}.$$

Factor out $\dfrac{1}{1+e^{-z}}$ from the numerator and denominator:

$$\sigma'(z) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z) \cdot \frac{1+e^{-z}-1}{1+e^{-z}} = \sigma(z)\big(1-\sigma(z)\big).$$

$$\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$$

This compact form makes the chain-rule calculations in gradient descent particularly clean.